

TRƯỜNG ĐẠI HỌC ...
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

ĐỀ TÀI: PHÂN TÍCH, THIẾT KẾ VÀ
XÂY DỰNG HỆ THỐNG QUẢN LÝ LỊCH
TRÌNH DU LỊCH CÁ NHÂN DỰA TRÊN
KIẾN TRÚC MVC

Sinh viên thực hiện:

- Vũ Thị Diệu Linh - 23015647
- Nguyễn Ngọc Linh - 23015739
- Vũ Quang Huy - 23016870

Giáo viên hướng dẫn: [Tên Giảng Viên]

Mục lục

MỞ ĐẦU	5
1 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	7
1.1 Bài toán thực tế và Nhu cầu hệ thống	7
1.1.1 Phát biểu bài toán	7
1.1.2 Nhu cầu giải quyết phần mềm	7
1.2 Mục tiêu và Phạm vi thiết kế	8
1.2.1 Mục tiêu hệ thống	8
1.2.2 Phạm vi thiết kế	8
1.3 Cơ sở lý thuyết về Kiến trúc và Bảo mật	8
1.3.1 Kiến trúc Model-View-Controller (MVC)	8
1.3.2 Mô hình kiểm soát truy cập và Lỗ hổng IDOR	8
1.4 Làm rõ công nghệ sử dụng và Hiện thực hóa mã nguồn	9
1.4.1 Framework PHP Laravel và Trình đóng gói Eloquent ORM	9
1.4.2 Thư viện kết xuất DomPDF và Cơ chế Render tài nguyên	9
1.4.3 Cơ chế nhúng bản đồ động tĩnh (Google Maps Embed URL Encoding)	10
1.4.4 Kiến trúc Frontend tối giản với Tailwind CSS và Alpine.js	10
2 PHÂN TÍCH HỆ THỐNG	11
2.1 Use case	11
2.2 Chức năng hệ thống	12
2.2.1 Phân hệ Xác thực và Định danh (Authentication)	12
2.2.2 Phân hệ Quản lý Địa điểm và Danh mục	12
2.2.3 Phân hệ Lịch trình (Itineraries)	12
2.3 Thiết kế CSDL	12
2.4 Sơ đồ luồng xử lý	14
3 THIẾT KẾ VÀ XÂY DỰNG	16
3.1 Kiến trúc Laravel MVC	16
3.1.1 Thành phần Model (Mô hình dữ liệu)	16
3.1.2 Thành phần View (Giao diện hiển thị)	16
3.1.3 Thành phần Controller (Bộ điều khiển logic)	17
3.1.4 Luồng hoạt động vòng đời (Request Lifecycle)	17
3.2 Các module chức năng	17
3.2.1 Module Xác thực và Phân quyền (Authentication & Authorization)	17
3.2.2 Module Quản lý Danh mục và Địa điểm (CRUD Core)	18
3.2.3 Module Xây dựng Lịch trình và Xử lý Bảng trung gian (Pivot Module)	18
3.2.4 Module Tích hợp và Kết xuất (Integration & Export)	19
3.3 Giao diện hệ thống	19
3.3.1 Triết lý thiết kế và Công nghệ UI	19
3.3.2 Trải nghiệm người dùng (UX) tại các trang trọng tâm	20
4 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	21
4.1 Kết quả thực nghiệm	21
4.2 Demo giao diện hệ thống	21
4.2.1 Bảng điều khiển (Dashboard)	21
4.2.2 Phân hệ Quản lý Địa điểm	21
4.2.3 Phân hệ Quản lý Tài khoản (Dành cho Admin)	22

4.2.4	Phân hệ Kiểm duyệt Lịch trình (Dành cho Admin)	23
4.3	Đánh giá ưu và nhược điểm	23
4.4	Phân công và đánh giá mức độ hoàn thành công việc	24
KẾT LUẬN VÀ KIẾN NGHỊ		25
TÀI LIỆU THAM KHẢO		26

Danh sách bảng

4.1	Bảng phân công nhiệm vụ và đánh giá đóng góp	24
-----	--	----

Danh sách hình vẽ

2.1	Biểu đồ Use case tổng quát của hệ thống	11
2.2	Sơ đồ mô hình quan hệ cơ sở dữ liệu (ERD)	13
2.3	Sơ đồ luồng xử lý xác thực và phân quyền (Authorization Workflow)	15
4.1	Giao diện Bảng điều khiển (Dashboard) tổng quan	22
4.2	Giao diện Quản lý địa điểm với cơ chế phân quyền hiển thị nút thao tác	22
4.3	Giao diện Quản lý Tài khoản người dùng của Quản trị viên	23
4.4	Giao diện Kiểm duyệt Lịch trình Cộng đồng của Quản trị viên	23

MỞ ĐẦU

1. Tổng quan tình hình nghiên cứu

Trong kỷ nguyên công nghệ 4.0, sự phát triển của Internet vạn vật (IoT) và dữ liệu lớn (Big Data) đã tạo ra bước nhảy vọt cho ngành công nghiệp không khói - Du lịch thông minh (Smart Tourism) [1]. Sự bùng nổ của các thiết bị di động và nhu cầu cá nhân hóa trải nghiệm du lịch đòi hỏi những nền tảng công nghệ mạnh mẽ để hỗ trợ người dùng. Hiện nay, trên thị trường có nhiều nền tảng lớn như TripAdvisor, Traveloka tập trung vào việc đặt vé và phòng khách sạn, nhưng lại thiếu đi các công cụ chuyên sâu giúp người dùng cá nhân hóa từng mốc thời gian (timeline) di chuyển một cách độc lập và trực quan [2].

2. Lý do chọn đề tài

Qua quá trình khảo sát thực tế, nhóm nhận thấy du khách tự túc thường phải sử dụng các công cụ thủ công như Microsoft Excel, Word hoặc Google Keep để ghi chép lịch trình. Điều này dẫn đến khó khăn trong việc quản lý, chia sẻ và theo dõi trực quan vị trí địa lý. Nhằm giải quyết điểm nghẽn này, nhóm quyết định thực hiện đề tài **"Xây dựng hệ thống quản lý lịch trình du lịch cá nhân"**. Ứng dụng cung cấp một giải pháp "All-in-one", từ khâu thu thập địa điểm, xây dựng dòng thời gian, cho đến xuất báo cáo PDF và chia sẻ công khai.

3. Mục tiêu nghiên cứu

Mục tiêu cốt lõi của đề tài là nghiên cứu quy trình phát triển một Web Application hoàn chỉnh dựa trên các tiêu chuẩn công nghiệp hiện đại. Các mục tiêu cụ thể bao gồm:

- Nghiên cứu và áp dụng mô hình kiến trúc Model-View-Controller (MVC) thông qua framework Laravel.
- Xây dựng hệ thống quản lý cơ sở dữ liệu quan hệ (RDBMS) với các ràng buộc khóa ngoại chặt chẽ.
- Phát triển cơ chế bảo mật, kiểm soát truy cập (Access Control) chống lại các lỗ hổng phổ biến như IDOR.
- Tích hợp các công cụ hỗ trợ người dùng: Google Maps (hiển thị vị trí) và DomPDF (xuất tệp).

4. Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Khung làm việc (Framework) Laravel, hệ quản trị CSDL MySQL, giao diện người dùng với Tailwind CSS.
- **Phạm vi nghiên cứu:** Phát triển phân hệ người dùng (tạo, chia sẻ lịch trình) và phân hệ quản trị viên (kiểm duyệt, quản lý danh mục) trên nền tảng Web.

5. Đóng góp chính của nghiên cứu

Nghiên cứu mang lại một nguyên mẫu (prototype) phần mềm hoạt động trơn tru với

độ bảo mật cao. Sản phẩm chứng minh tính khả thi của việc kết hợp các công cụ mã nguồn mở để tạo ra một hệ sinh thái phần mềm đáp ứng nhu cầu khắt khe của quy trình Software Engineering.

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

1.1 Bài toán thực tế và Nhu cầu hệ thống

1.1.1 Phát biểu bài toán

Trong kỷ nguyên số hóa ngành du lịch, nhu cầu tự lên kế hoạch hành trình (Cá nhân hóa lịch trình - Personalized Itinerary Planning) đang trở thành xu hướng chủ đạo của du khách tự túc. Tuy nhiên, một bài toán thực tế đặt ra là sự thiếu hụt các công cụ chuyên dụng giúp quản lý tập trung và đồng bộ dòng thời gian (Timeline) di chuyển.

Người dùng thường đối mặt với các vấn đề:

- **Phân tán thông tin:** Dữ liệu về địa điểm, thời gian ghé thăm, ghi chú lộ trình bị chia nhỏ trên nhiều nền tảng không đồng bộ (Excel, giấy ghi chú, tin nhắn).
- **Thiếu trực quan hình học:** Không kết nối được địa chỉ văn bản thuần túy với bản đồ định vị trực quan trong cùng một giao diện hiển thị.
- **Rủi ro về an toàn thông tin:** Các hệ thống lưu trữ mở hoặc 1 lôm cô m dễ mắc phải các lỗ hổng phân quyền dữ liệu, cho phép người dùng này can thiệp, chỉnh sửa trái phép lộ trình của người dùng khác.

1.1.2 Nhu cầu giải quyết phần mềm

Để giải quyết triệt để bài toán trên, hệ thống cần phải hiện thực hóa một mô hình kiến trúc phần mềm đáp ứng ba trục cốt lõi:

1. **Trục Dữ liệu quan hệ bền vững:** Tổ chức cấu trúc CSDL tối ưu, xử lý mượt mà mối quan hệ Nhiều - Nhiều giữa các chuyến đi và tập hợp địa điểm mà không làm phình to hoặc trùng lặp dữ liệu tĩnh.
2. **Trục Bảo mật phân quyền nghiêm ngặt:** Đảm bảo tính toàn vẹn dữ liệu thông qua các bộ lọc HTTP Request, chặn đứng các hành vi khai thác lỗ hổng logic từ phía Client.
3. **Trục Tiện ích kết xuất linh hoạt:** Cung cấp khả năng biên dịch tài nguyên động thành các định dạng tĩnh có thể lưu trữ ngoại tuyến (Offline PDF) và cơ chế định vị không tốn tài nguyên hệ thống.

1.2 Mục tiêu và Phạm vi thiết kế

1.2.1 Mục tiêu hệ thống

Hệ thống hướng tới việc xây dựng một Web Application hoàn chỉnh, an toàn và đạt hiệu năng cao. Mục tiêu kỹ thuật cụ thể bao gồm:

- **Về mặt kiến trúc:** Triển khai thành công mô hình MVC chuẩn công nghiệp, tách biệt hoàn toàn mã nguồn xử lý logic và mã nguồn hiển thị giao diện.
- **Về mặt bảo mật:** Triển khai mô hình kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC) và bảo vệ thực thể ở mức định danh dòng (Row-level protection).
- **Về mặt UX/UI:** Đạt độ phản hồi giao diện dưới 2 giây, tương thích Responsive 100% trên đa thiết bị nhờ cơ chế CSS động.

1.2.2 Phạm vi thiết kế

Dự án tập trung giải quyết bài toán quản lý lịch trình trong phạm vi nền tảng Web phía máy chủ (Server-side rendering), tối ưu hóa cấu trúc mã nguồn thông qua Framework Laravel và các thư viện hỗ trợ, không mở rộng sang hạ tầng Microservices phức tạp nhằm đảm bảo tính gọn nhẹ và tập trung vào hiệu năng lõi.

1.3 Cơ sở lý thuyết về Kiến trúc và Bảo mật

1.3.1 Kiến trúc Model-View-Controller (MVC)

Mẫu thiết kế MVC là nền tảng cốt lõi của nghiên cứu này. Logic vận hành dựa trên nguyên lý chia nhỏ ứng dụng thành 3 tầng chức năng biệt lập [3]:

$$\text{Request} \longrightarrow \text{Controller} \rightleftharpoons \text{Model (Database)} \longrightarrow \text{View} \longrightarrow \text{Response} \quad (1.1)$$

Sự cô lập này mang lại lợi thế: khi giao diện (View) thay đổi cấu trúc hiển thị, logic xử lý dữ liệu tại Controller và Model hoàn toàn không bị ảnh hưởng, giúp giảm thiểu tối đa hiện tượng xung đột mã nguồn khi phát triển nhóm.

1.3.2 Mô hình kiểm soát truy cập và Lỗ hổng IDOR

Hệ thống xử lý bài toán bảo mật dựa trên hai tầng rào cản:

- **Authentication (Xác thực):** Sử dụng cơ chế quản lý trạng thái Session dựa trên Cookie định danh để xác minh danh tính người dùng qua từng HTTP Request gửi lên.

- **Authorization (Phân quyền):** Áp dụng mô hình RBAC [4]. Tuy nhiên, để giải quyết triệt để lỗ hổng **IDOR (Insecure Direct Object Reference)** — trường hợp kẻ tấn công thay đổi tham số ID (ví dụ: `/locations/5/edit` thành `/locations/6/edit`) để thao tác dữ liệu trái phép — hệ thống bắt buộc phải kiểm tra quyền sở hữu thực thể ở mức logic tầng xử lý. Một bản ghi chỉ được cho phép cập nhật khi và chỉ khi:

$$\text{User}_{\text{current}} = \text{Owner}_{\text{Data}} \quad \vee \quad \text{User}_{\text{role}} = \text{'admin'}$$
 (1.2)

1.4 Làm rõ công nghệ sử dụng và Hiện thực hóa mã nguồn

1.4.1 Framework PHP Laravel và Trình đóng gói Eloquent ORM

Laravel được chọn làm nền tảng phát triển phía máy chủ nhờ vào hệ sinh thái công cụ mạnh mẽ và khả năng bảo mật mặc định cao [5].

- **Cơ chế bảo vệ CSRF và XSS:** Laravel tự động bắt buộc đính kèm thẻ mẫu mã hóa độc quyền (CSRF Token) vào mỗi Request gửi lên từ Form để chống lại các cuộc tấn công giả mạo. Đồng thời, Blade Engine mặc định sử dụng cơ chế escape dữ liệu `{{ }}` để vô hiệu hóa hoàn toàn mã độc script từ người dùng gửi lên.
- **Bản chất xử lý Pivot trong Eloquent ORM:** Đối với bài toán Nhiều - Nhiều giữa chuyển đi và địa điểm, thay vì lập trình các câu lệnh SQL JOIN thủ công phức tạp dễ gây nghẽn hệ thống, Eloquent cung cấp cơ chế ánh xạ quan hệ trừu tượng thông qua hàm `belongsToMany()`. Hệ thống tận dụng tối đa chất xám của cấu trúc này để nhúng thêm dữ liệu động (`visit_time`, `note`) vào bảng trung gian, cho phép biến bảng liên kết vật lý thành một thực thể chứa ngữ cảnh thời gian (Timeline Data).

1.4.2 Thư viện kết xuất DomPDF và Cơ chế Render tài nguyên

Để hiện thực hóa chức năng in ấn và lưu trữ lịch trình, thư viện DomPDF đóng vai trò bộ chuyển đổi mã nguồn động (Dynamic HTML-to-PDF Converter).

- **Nguyên lý hoạt động:** DomPDF tiếp nhận luồng dữ liệu HTML thô từ View, xây dựng một cây mô hình đối tượng tài liệu (DOM Tree) nội bộ, sau đó áp dụng một công cụ dàn trang (Layout Engine) dựa trên CSS nội tuyến để tính toán tọa độ vật lý của từng khối văn bản trên khổ giấy A4.
- **Xử lý Byte Stream:** Controller sẽ bọc dữ liệu, gọi thư viện đóng gói thành một chuỗi nhị phân (Binary Byte Stream) và đẩy ngược về HTTP Header với định dạng

Content-Type: application/pdf, ép trình duyệt client thực thi hành động tải xuống tệp tin mà không cần lưu trữ tệp vật lý tạm thời trên đĩa cứng server, giúp tối ưu hóa dung lượng bộ nhớ.

1.4.3 Cơ chế nhúng bản đồ động tĩnh (Google Maps Embed URL Encoding)

Hệ thống giải quyết bài toán định vị địa lý bằng giải pháp nhúng cấu trúc Iframe thông qua kỹ thuật mã hóa chuỗi URL (URL Encoding).

- **Logic giải thuật:** Toàn bộ chuỗi địa chỉ văn bản do người dùng nhập vào (chứa các ký tự đặc biệt, dấu cách, tiếng Việt có dấu) sẽ được đi qua hàm mã hóa cấu trúc mạng `urlencode()`. Hàm này chuyển đổi các ký tự không hợp lệ thành các chuỗi mã hex (ví dụ: dấu cách thành `%20`).
- **Tối ưu hiệu năng:** Chuỗi mã hóa này được nối thẳng vào endpoint nhúng chính thức của Google Maps. Trình duyệt phía Client sẽ chịu trách nhiệm gửi request trực tiếp đến máy chủ Google để render bản đồ trong vùng Iframe biệt lập. Giải pháp này giúp hệ thống hoạt động hoàn toàn miễn phí, không bị ràng buộc bởi giới hạn lượt gọi (Quotas) và loại bỏ hoàn toàn gánh nặng xử lý dữ liệu bản đồ trên máy chủ của ứng dụng.

1.4.4 Kiến trúc Frontend tối giản với Tailwind CSS và Alpine.js

Để tối ưu hóa giao diện hiển thị thích ứng (Responsive UI) trên đa thiết bị, hệ thống ứng dụng mô hình phát triển Frontend hiện đại, tối giản hóa dung lượng tải trang.

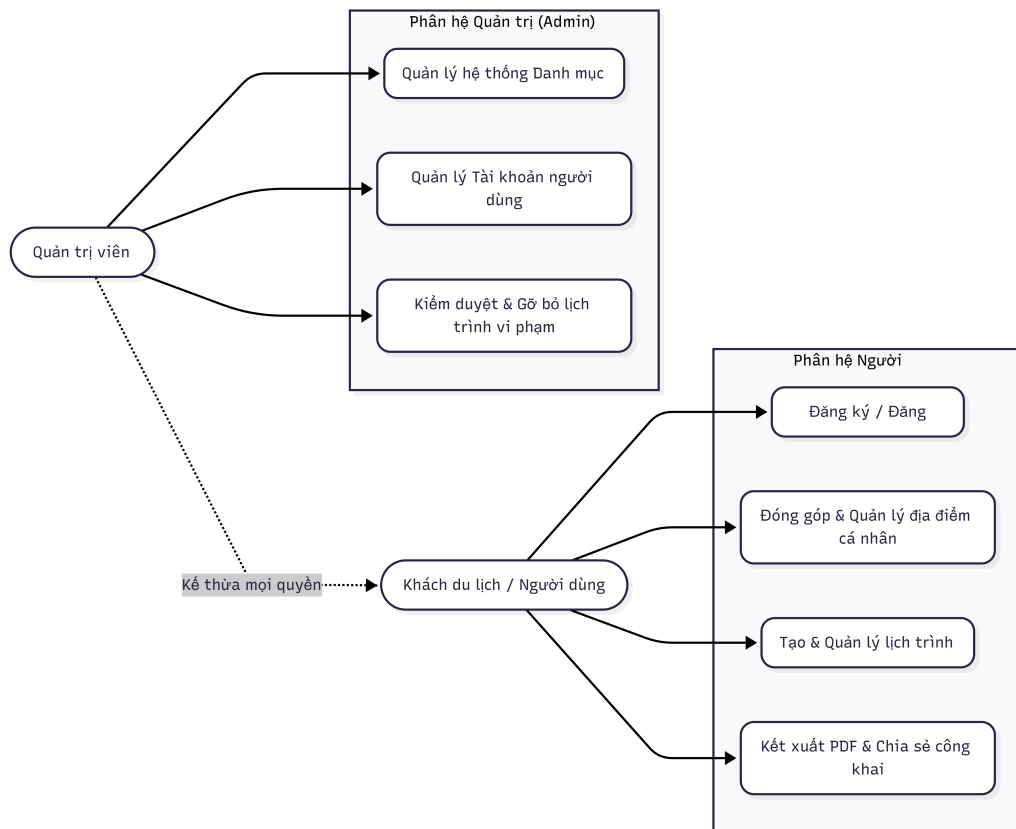
- **Tailwind CSS và cơ chế Biên dịch JIT (Just-In-Time):** Không sử dụng các bộ CSS dựng sẵn công kênh, Tailwind CSS kết hợp với trình đóng gói Vite sẽ quét toàn bộ các file Blade View, chỉ giữ lại và biên dịch những class thực sự được sử dụng thành một file CSS duy nhất có dung lượng cực nhỏ (chỉ vài chục KB), giúp giảm thiểu thời gian tải trang xuống mức tối đa.
- **Alpine.js và Cơ chế Phản hồi DOM nội tại:** Đối với các tương tác UI động phía Client (như ẩn hiện thanh menu, đóng mở hộp thoại xác nhận xóa tài khoản), hệ thống sử dụng Alpine.js. Thư viện này chèn trực tiếp các thuộc tính phản xạ (reactive attributes như `x-data`, `x-show`) thẳng vào cây DOM hiện tại, giúp xử lý tương tác mượt mà ngay tại trình duyệt mà không cần phải tiêu tốn tài nguyên tải các framework nặng nề như React hay Vue.

PHÂN TÍCH HỆ THỐNG

2.1 Use case

Mô hình Use case (Biểu đồ ca sử dụng) giúp xác định rõ các tác nhân (Actors) tham gia vào hệ thống và những hành động (Use cases) mà họ có thể thực hiện. Dựa trên khảo sát yêu cầu thực tế, hệ thống có hai tác nhân chính: **Người dùng (User)** và **Quản trị viên (Admin)**.

- **Người dùng (User):** Là khách du lịch hoặc cá nhân có nhu cầu lưu trữ và lên kế hoạch. Các Use case bao gồm: Đăng ký/Đăng nhập, Quản lý địa điểm cá nhân (Thêm, sửa, xóa bài của mình), Tạo và quản lý lịch trình, Thêm địa điểm vào lịch trình, Xuất lịch trình ra tệp PDF, và Chia sẻ liên kết công khai.
- **Quản trị viên (Admin):** Kế thừa mọi Use case của Người dùng, đồng thời có thêm các Use case đặc quyền: Xem Dashboard thống kê, Quản lý Danh mục lỗi, Quản lý toàn bộ tài khoản người dùng, và Kiểm duyệt (Xóa) các lịch trình/địa điểm vi phạm nội quy.



Hình 2.1: Biểu đồ Use case tổng quát của hệ thống

2.2 Chức năng hệ thống

Từ các Use case đã phân tích, hệ thống được thiết kế và chia thành các phân hệ chức năng (Functional Modules) cụ thể như sau:

2.2.1 Phân hệ Xác thực và Định danh (Authentication)

Đảm nhiệm việc đăng ký tài khoản mới và xác thực người dùng. Mật khẩu được băm (hashing) một chiều bằng thuật toán Bcrypt. Hệ thống sử dụng Session để duy trì trạng thái đăng nhập và phân biệt phân quyền (Role) giữa Admin và User.

2.2.2 Phân hệ Quản lý Địa điểm và Danh mục

- **Danh mục (Categories):** Cung cấp tính năng nhóm các địa điểm theo chủ đề (Ví dụ: Biển, Rừng, Di tích). Chỉ Admin mới có quyền CRUD (Thêm, Đọc, Sửa, Xóa) danh mục.
- **Địa điểm (Locations):** Cho phép người dùng tải lên hình ảnh, tên, địa chỉ và mô tả chi tiết. Hệ thống tích hợp Iframe Google Maps dựa trên địa chỉ lưu trữ. Tính năng Sửa/Xóa được bảo vệ nghiêm ngặt bằng logic đối chiếu ID chủ sở hữu.

2.2.3 Phân hệ Lịch trình (Itineraries)

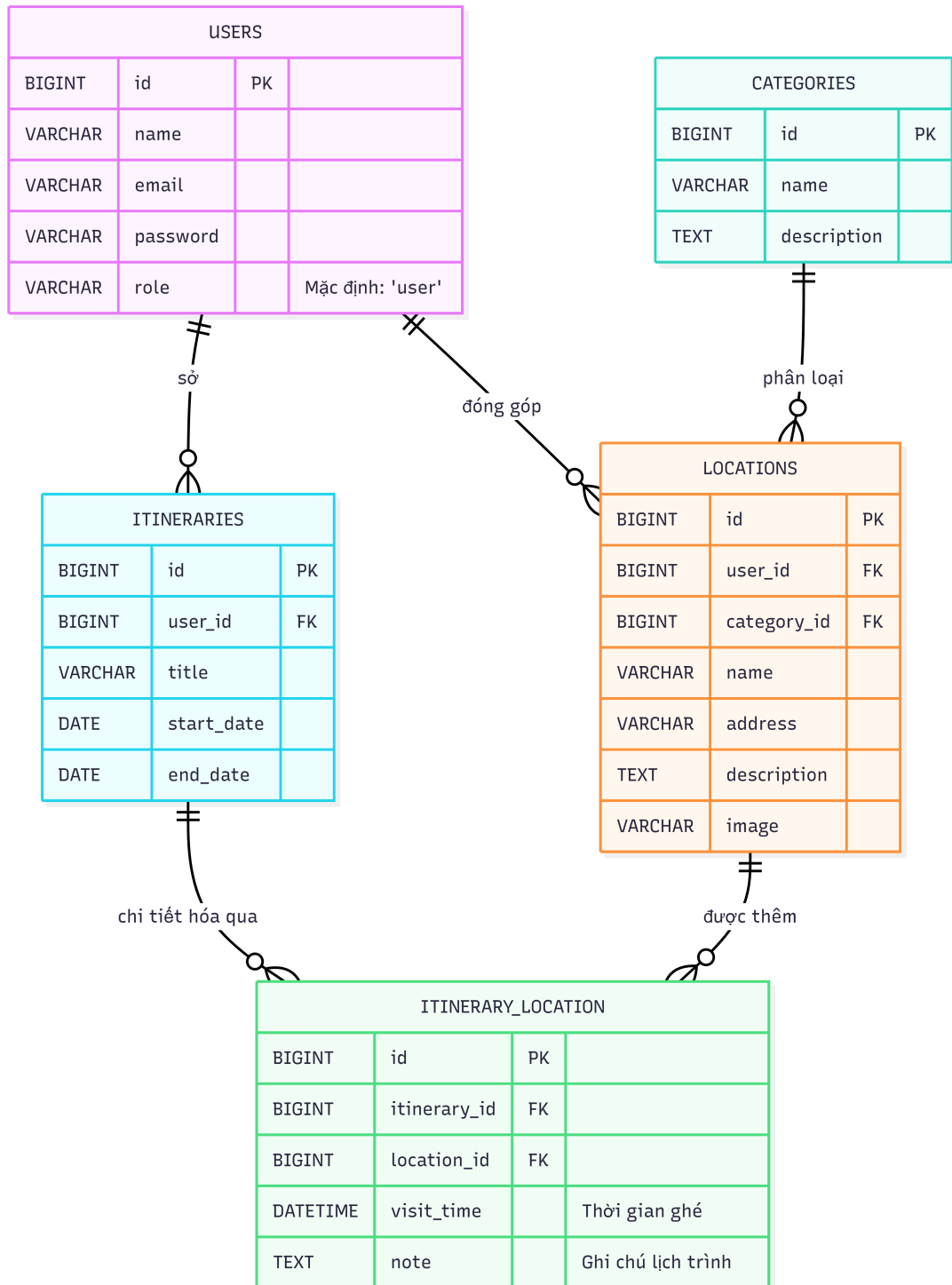
Đây là chức năng hạt nhân của dự án. Người dùng tạo các chuyến đi với ngày bắt đầu và kết thúc. Sau đó, họ có thể chọn các địa điểm từ kho dữ liệu chung, đưa vào lịch trình của mình, đồng thời thiết lập chính xác *thời gian ghé thăm (visit_time)* và *ghi chú (note)* tạo thành một dòng thời gian (Timeline) hoàn chỉnh. Chức năng kết xuất (Export) cho phép biến toàn bộ Timeline này thành file PDF hoặc liên kết chia sẻ (Public Link).

2.3 Thiết kế CSDL

Hệ thống sử dụng hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) MySQL. Mô hình dữ liệu được thiết kế đạt chuẩn hóa 3NF, bao gồm 5 thực thể (Entities) cốt lõi có liên kết ràng buộc khóa ngoại (Foreign Keys) chặt chẽ:

1. **users:** Lưu thông tin tài khoản (*id, name, email, password, role*).
2. **categories:** Lưu các phân loại danh mục (*id, name, description*).
3. **locations:** Lưu chi tiết địa điểm (*id, user_id, category_id, name, address, description, image*).
4. **itineraries:** Lưu siêu dữ liệu chuyến đi (*id, user_id, title, start_date, end_date*).

5. **itinerary_location**: Bảng trung gian (Pivot Table) giải quyết mối quan hệ Nhiều-Nhiều (N-N) giữa **itineraries** và **locations**. Đặc biệt, bảng này được mở rộng lưu trữ thêm trường **visit_time** và **note** để biểu diễn trực thời gian thực tế của chuyến đi.

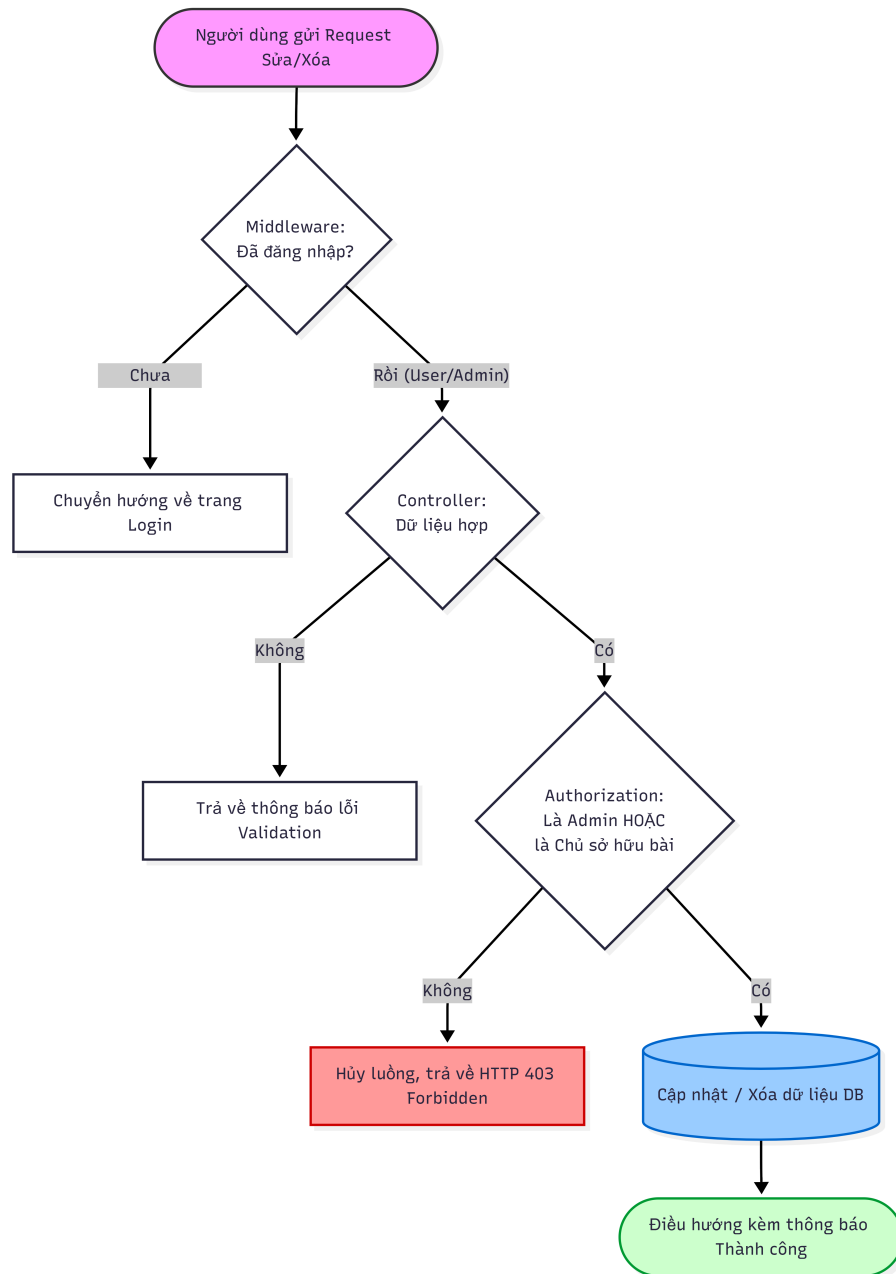


Hình 2.2: Sơ đồ mô hình quan hệ cơ sở dữ liệu (ERD)

2.4 Sơ đồ luồng xử lý

Để đảm bảo tính bảo mật và toàn vẹn dữ liệu, mọi tương tác thay đổi dữ liệu (Cập nhật hoặc Xóa) đều phải trải qua luồng xử lý (Workflow) xác thực hai lớp. Dưới đây là luồng xử lý tiêu biểu cho chức năng **Chỉnh sửa/Xóa Địa điểm**:

- **Bước 1 (Middleware Check):** Yêu cầu HTTP từ Client gửi lên bắt buộc phải đi qua Middleware `auth` để kiểm tra trạng thái đăng nhập. Nếu chưa đăng nhập, điều hướng về trang Login.
- **Bước 2 (Controller Validation):** Xác thực định dạng dữ liệu đầu vào (không rỗng, đúng độ dài, đúng định dạng file ảnh) thông qua `Request Validation`.
- **Bước 3 (Authorization Logic):** Hệ thống trích xuất `user_id` của bản ghi địa điểm và so sánh với `Auth::id()` của người đang thực hiện. Nếu không trùng khớp, tiếp tục kiểm tra xem `role` của người đó có phải là `admin` hay không.
- **Bước 4 (Execution or Abort):** Nếu một trong hai điều kiện trên đúng, cho phép thao tác CSDL. Nếu sai, lập tức chặn quyền truy cập và trả về màn hình lỗi **HTTP 403 Forbidden**.



Hình 2.3: Sơ đồ luồng xử lý xác thực và phân quyền (Authorization Workflow)

THIẾT KẾ VÀ XÂY DỰNG

3.1 Kiến trúc Laravel MVC

Hệ thống được thiết kế và phát triển dựa trên nền tảng framework Laravel, tuân thủ nghiêm ngặt mô hình kiến trúc Model-View-Controller (MVC). Việc áp dụng MVC giúp tách biệt hoàn toàn giữa logic xử lý nghiệp vụ (Business Logic), thao tác cơ sở dữ liệu và giao diện hiển thị, từ đó tối ưu hóa quá trình phát triển, mở rộng và bảo trì mã nguồn.

3.1.1 Thành phần Model (Mô hình dữ liệu)

Trong Laravel, Model đóng vai trò giao tiếp trực tiếp với cơ sở dữ liệu thông qua Eloquent ORM (Object-Relational Mapping). Mỗi bảng trong cơ sở dữ liệu được ánh xạ thành một lớp (Class) Model độc lập.

- **Định nghĩa thực thể:** Hệ thống bao gồm các Model chính quản lý dữ liệu lõi như `User`, `Category`, `Location`, và `Itinerary`.
- **Thiết lập quan hệ (Relationships):** Các mối quan hệ thực thể (1-N, N-N) được định nghĩa bằng các phương thức Eloquent. Điển hình là mối quan hệ Nhiều - Nhiều (Many-to-Many) giữa Lịch trình (`Itinerary`) và Địa điểm (`Location`). Trong Model `Itinerary`, phương thức `locations()` sử dụng lệnh `belongsToMany` để liên kết qua bảng trung gian `itinerary_location`. Đặc biệt, hệ thống sử dụng hàm `withPivot('visit_time', 'note')` để truy xuất các thuộc tính động của tiến trình thời gian (timeline) lưu tại bảng trung gian.

3.1.2 Thành phần View (Giao diện hiển thị)

Thành phần View chịu trách nhiệm biên dịch dữ liệu thành giao diện tương tác cho người dùng cuối. Hệ thống sử dụng **Blade Template Engine** - công cụ render giao diện mặc định, an toàn và mạnh mẽ của Laravel.

- **Kế thừa Layout (Template Inheritance):** Toàn bộ giao diện được xây dựng trên một layout gốc (`app.blade.php`). Các trang con sử dụng chỉ thị `@extends` và `@section` để tái sử dụng mã HTML/CSS (như Header, Footer, Navigation Bar), giúp mã nguồn Frontend ngắn gọn và nhất quán.
- **Tích hợp Frontend hiện đại:** View được tích hợp trực tiếp với framework **Tailwind CSS** thông qua trình đóng gói Vite, mang lại khả năng xây dựng giao diện linh hoạt theo phong cách Utility-first. Các logic xử lý sự kiện phía client (như đóng mở modal, dropdown, ẩn hiện menu) được xử lý nhẹ nhàng bằng thư viện **Alpine.js**.

3.1.3 Thành phần Controller (Bộ điều khiển logic)

Controller đóng vai trò là "nhạc trưởng", tiếp nhận các HTTP Request từ Client, xử lý logic nghiệp vụ và trả về HTTP Response (View hoặc chuyển hướng Redirect).

- **Cấu trúc Resource Controller:** Quản lý các thao tác CRUD (Create, Read, Update, Delete) cho từng đối tượng một cách chuẩn hóa. Ví dụ: `LocationController` chứa đầy đủ các phương thức `index()`, `create()`, `store()`, `edit()`, `update()`, `destroy()`.
- **Xử lý Form Request & Validation:** Trước khi dữ liệu được truyền xuống Model để lưu vào DB, Controller sử dụng cơ chế Validation của Laravel để kiểm tra tính hợp lệ (ví dụ: bắt buộc nhập tên, định dạng ảnh `mimes:jpg,png`, giới hạn dung lượng, chống rỗng).
- **Bảo mật luồng (Authorization Logic):** Tại các hàm có tính phá hủy dữ liệu như `update()` và `destroy()`, Controller thực thi kiểm tra quyền sở hữu bài viết: `if (Auth::user()->role !== 'admin' && $location->user_id !== Auth::id()) { abort(403); }`, giúp khóa chặt lỗ hổng bảo mật truy cập trái phép.

3.1.4 Luồng hoạt động vòng đời (Request Lifecycle)

Một luồng xử lý (Data flow) hoàn chỉnh của hệ thống diễn ra theo các bước nghiêm ngặt sau: Người dùng gửi yêu cầu qua URL → Router (`routes/web.php`) tiếp nhận và đưa qua màn lọc Middleware (ví dụ: `auth` để check đăng nhập) → Chuyển tiếp đến Controller tương ứng → Controller tương tác với Model để thực thi truy vấn Eloquent tới CSDL MySQL → Controller đóng gói dữ liệu (Compact) và gửi sang View → Blade Engine biên dịch View thành HTML hoàn chỉnh và trả về trình duyệt của người dùng.

3.2 Các module chức năng

Hệ thống được chia thành 4 module chức năng lõi, hoạt động độc lập nhưng liên kết chặt chẽ với nhau thông qua cơ sở dữ liệu và các luồng Service của Laravel.

3.2.1 Module Xác thực và Phân quyền (Authentication & Authorization)

- **Cơ chế Xác thực (Authentication):** Module này quản lý vòng đời phiên đăng nhập (Session Lifecycle). Khi người dùng đăng ký, mật khẩu sẽ được băm một chiều bằng thuật toán **Bcrypt** trước khi lưu vào CSDL, đảm bảo an toàn tuyệt đối ngay cả khi dữ liệu bị rò rỉ.
- **Cơ chế Phân quyền (Authorization):** Hệ thống định nghĩa một `AdminMiddleware` tùy chỉnh. Middleware này đóng vai trò như một màn lọc bảo vệ các Route nhạy

cảm (như trang Quản lý Người dùng, Kiểm duyệt Lịch trình). Nếu thuộc tính `role` của User trong Session không phải là `'admin'`, hệ thống tự động ngắt luồng và trả về mã lỗi **HTTP 403 (Forbidden)**.

- **Chống lỗ hổng IDOR:** Trong các hàm sửa/xóa địa điểm cá nhân, Module thực thi logic kiểm tra chéo: `Auth::id() === $location->user_id`. Điều này ngăn chặn triệt để hành vi người dùng cố tình thay đổi tham số ID trên URL để can thiệp vào dữ liệu của người khác.

3.2.2 Module Quản lý Danh mục và Địa điểm (CRUD Core)

- **Quản lý hình ảnh (File Upload Handling):** Khi người dùng tải ảnh địa điểm lên, Module sử dụng `Storage Facade` của Laravel để kiểm tra tính hợp lệ (MIME type, dung lượng). Sau đó, file ảnh được đổi tên bằng chuỗi Hash ngẫu nhiên để tránh trùng lặp, lưu trữ vào thư mục `public/storage/locations` và lưu đường dẫn vật lý vào CSDL.
- **Tối ưu truy vấn (Eager Loading):** Khi hiển thị danh sách địa điểm, Module sử dụng kỹ thuật Eager Loading để giải quyết bài toán **N+1 Query**, giúp giảm thiểu số lượng câu lệnh SQL truy vấn xuống DB, từ đó tăng tốc độ tải trang đáng kể:

```
$locations = Location::with('category')->get();
```

3.2.3 Module Xây dựng Lịch trình và Xử lý Bảng trung gian (Pivot Module)

Đây là Module có độ phức tạp thuật toán cao nhất trong hệ thống, quản lý trục thời gian (Timeline) của chuyến đi.

- **Ghi dữ liệu vào Pivot Table:** Khi người dùng chọn một Địa điểm để thêm vào Lịch trình, Module không tạo bảng mới mà dùng phương thức `attach()` của Eloquent. Dữ liệu được đẩy vào bảng trung gian kèm theo mảng thuộc tính động:

```
$itinerary->locations()->attach($locationId, [
    'visit_time' => $request->time,
    'note' => $request->note
]);
```

- **Trích xuất và Sắp xếp Timeline:** Khi render chi tiết chuyến đi, hệ thống lấy dữ liệu ra và bắt buộc phải sắp xếp theo trình tự thời gian ghé thăm để tạo thành một lộ trình hợp lý. Khối lệnh xử lý:

```
$locations = $itinerary->locations()
```

```

->withPivot('visit_time', 'note')
->orderBy('itinerary_location.visit_time', 'asc')
->get();

```

3.2.4 Module Tích hợp và Kết xuất (Integration & Export)

- **Tích hợp Google Maps Iframe động:** Thay vì sử dụng Google Maps API dạng JavaScript (cần khóa API và có giới hạn số lượt gọi), Module áp dụng giải pháp nhúng Iframe trực tiếp. Địa chỉ của địa điểm được mã hóa thông qua hàm `urlencode()` của PHP và nối thẳng vào URL nguồn của Google Maps, giúp render bản đồ ngay lập tức với độ chính xác cao và hoàn toàn miễn phí.
- **Kết xuất tệp PDF (DomPDF):** Sử dụng thư viện `barryvdh/laravel-dompdf`. Khi người dùng yêu cầu tải xuống, Module sẽ đóng gói toàn bộ dữ liệu của Lịch trình (bao gồm cả Timeline đã sắp xếp), đưa vào một Blade View chuyên dụng được thiết kế bằng CSS nội tuyến. Thư viện sẽ parse mã HTML này và ép trình duyệt tải về qua phương thức:

```

return PDF::loadView('pdf.itinerary', compact('itinerary', 'locations'))
->download('lich-trinh.pdf');

```

3.3 Giao diện hệ thống

Giao diện hệ thống được thiết kế theo trường phái tối giản (Minimalism) kết hợp với các đường nét minh họa (Illustrative) hiện đại, nhằm mang lại trải nghiệm người dùng (UX) trực quan và tập trung tối đa vào nội dung cốt lõi là các địa điểm và lịch trình du lịch.

3.3.1 Triết lý thiết kế và Công nghệ UI

- **Mobile-first & Responsive:** Toàn bộ giao diện được xây dựng bằng framework Tailwind CSS theo hướng ưu tiên thiết bị di động (Mobile-first). Các thành phần giao diện (UI Components) như thanh điều hướng, thẻ bài viết (Cards), và lưới hình ảnh tự động co giãn, sắp xếp lại để hiển thị hoàn hảo trên đa dạng kích thước màn hình (Desktop, Tablet, Mobile).
- **Màu sắc và Thị giác:** Hệ thống sử dụng tông màu trung tính (trắng, xám nhạt) làm nền, kết hợp với các điểm nhấn màu sắc có độ tương phản cao cho các nút hành động chính (Call-to-Action) nhằm điều hướng sự chú ý của người dùng một cách tinh tế mà không gây rối mắt.

3.3.2 Trải nghiệm người dùng (UX) tại các trang trọng tâm

- **Trang Quản lý Lịch trình (Timeline View):** Thay vì liệt kê dữ liệu dưới dạng bảng tính nhàm chán, hệ thống biểu diễn các điểm đến dưới dạng dòng thời gian (Timeline). Các mốc thời gian được đánh dấu bằng các biểu tượng (Icons) và nhãn dán (Badges) sinh động, nối liền bằng trục dọc để du khách dễ dàng hình dung thứ tự di chuyển.
- **Tối ưu hóa phản hồi tương tác (Interactive Feedback):** Mọi thao tác thay đổi dữ liệu (Thêm, Sửa, Xóa) đều được hệ thống phản hồi tức thời thông qua các thông báo bật lên (Toast Notifications). Các biểu mẫu nhập liệu (Forms) được tích hợp cảnh báo lỗi trực tiếp (Inline Validation). Nếu người dùng bỏ trống hoặc nhập sai định dạng, hệ thống lập tức hiển thị cảnh báo viền đỏ dưới trường nhập liệu mà không cần tải lại toàn bộ trang.

KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Kết quả thực nghiệm

Hệ thống đã trải qua quá trình kiểm thử hộp đen (Black-box testing) trên các luồng người dùng độc lập: Admin, User A, và User B. Kết quả ghi nhận:

- **Tính toàn vẹn dữ liệu:** Hệ thống phân quyền hoạt động chính xác. Chức năng Sửa/Xóa chỉ hiển thị với chủ sở hữu bài viết hoặc Quản trị viên. Các nỗ lực truy cập trái phép qua URL đều bị chặn bởi Middleware và trả về lỗi HTTP 403.
- **Tính năng mở rộng:** Tính năng xuất file PDF và nhúng bản đồ Google Maps Iframe hoạt động mượt mà, không gây ảnh hưởng đến thời gian tải trang.
- **Bảo mật hệ thống:** Thông tin mật khẩu được mã hóa an toàn, các form nhập liệu đều vượt qua kiểm tra CSRF Token và Validation chống rỗng.

4.2 Demo giao diện hệ thống

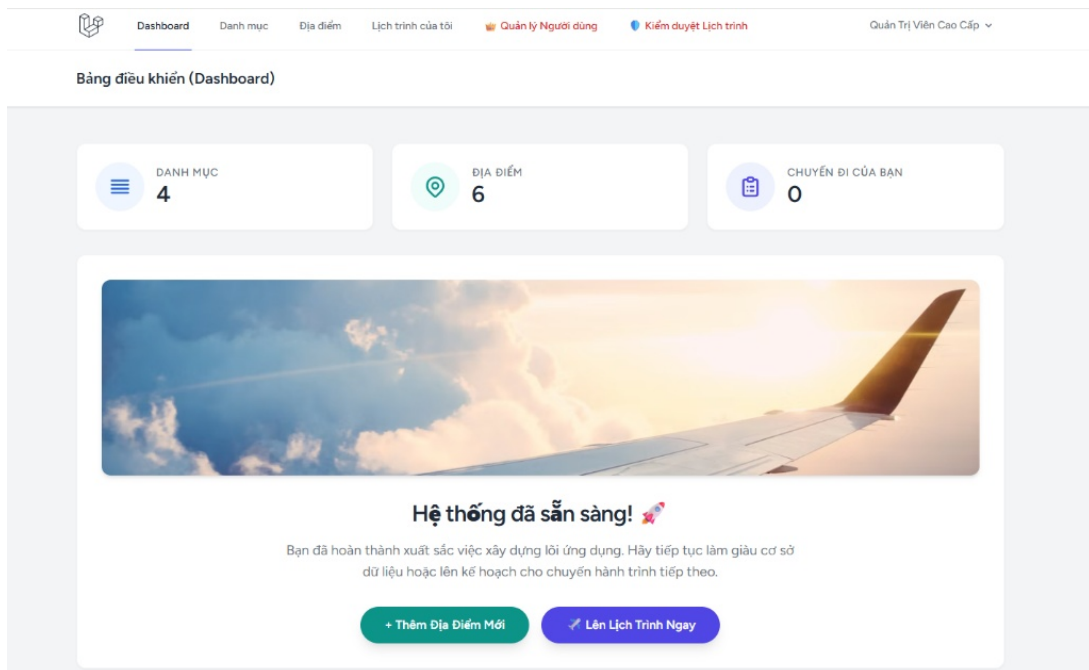
Để minh chứng cho các kết quả đạt được, dưới đây là hình ảnh thực tế (Demo) của một số phân hệ tính năng trọng tâm đã được triển khai và vận hành trên hệ thống:

4.2.1 Bảng điều khiển (Dashboard)

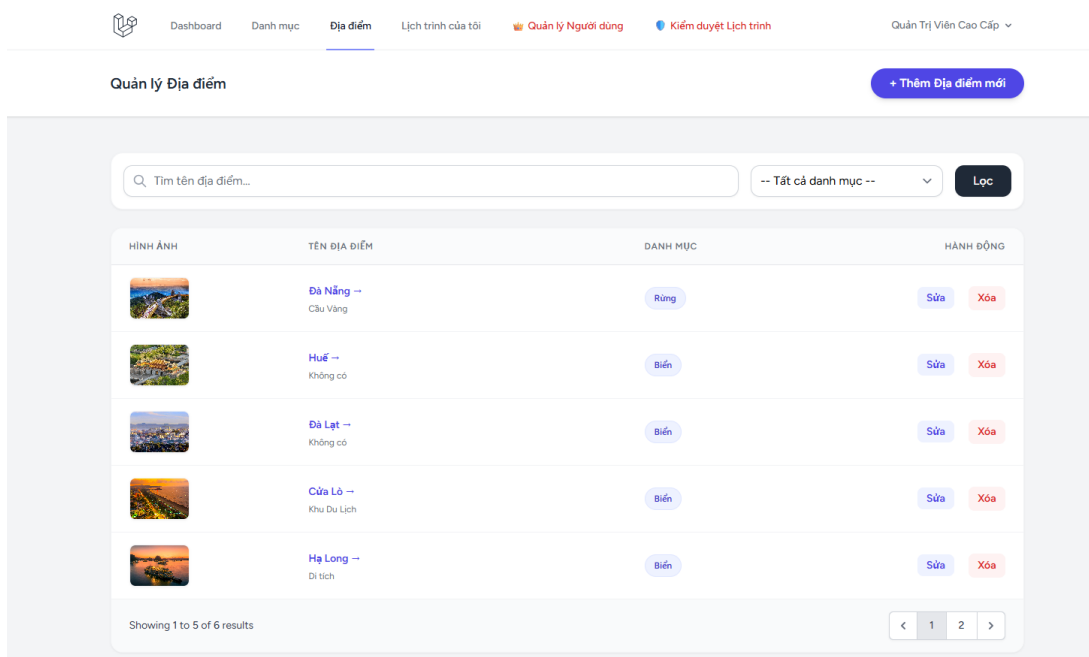
Bảng điều khiển là giao diện đầu tiên người dùng tiếp cận sau khi đăng nhập thành công. Giao diện cung cấp cái nhìn tổng quan bằng các thẻ thống kê (Analytics Cards) về số lượng Danh mục, Địa điểm hiện có trên hệ thống và số lượng Chuyển đi cá nhân đã tạo.

4.2.2 Phân hệ Quản lý Địa điểm

Phân hệ này hiển thị danh sách các địa điểm du lịch kèm theo hình ảnh thu nhỏ, tên địa điểm và danh mục tương ứng. Điểm nhấn về mặt logic phân quyền được thể hiện rõ: hệ thống tự động nhận diện chủ sở hữu để hiển thị các nút hành động **Sửa / Xóa**. Đối với các địa điểm do người khác đóng góp, hệ thống sẽ ẩn các nút thao tác này đối với người dùng thường nhằm bảo vệ dữ liệu.



Hình 4.1: Giao diện Bảng điều khiển (Dashboard) tổng quan



Hình 4.2: Giao diện Quản lý địa điểm với cơ chế phân quyền hiển thị nút thao tác

4.2.3 Phân hệ Quản lý Tài khoản (Dành cho Admin)

Đây là khu vực đặc quyền của Quản trị viên cấp cao. Hệ thống liệt kê toàn bộ định danh người dùng, địa chỉ email và vai trò (Role). Admin có thể thực thi quyền lực tối cao là **Xóa tài khoản** đối với các thành viên vi phạm tiêu chuẩn cộng đồng. Tài khoản Admin được hệ thống bảo vệ cứng bằng logic tránh tự xóa (trạng thái "Bất tử").

	Dashboard	Danh mục	Địa điểm	Lịch trình của tôi	Quản lý Người dùng	Kiểm duyệt Lịch trình	Quản Trị Viên Cao Cấp ▾
---	-----------	----------	----------	--------------------	---------------------------	-----------------------	-------------------------

Quản lý Tài khoản Người dùng				
ID	TÊN NGƯỜI DÙNG	EMAIL	VAI TRÒ	HÀNH ĐỘNG
#3	Quản Trị Viên Cao Cấp	admin@gmail.com	Admin	Bật/tắt
#4	Nguyễn Văn A	usera@gmail.com	User	Xóa tài khoản
#5	Trần Thị B	userb@gmail.com	User	Xóa tài khoản
#2	Huy	huyvu@gmail.com	User	Xóa tài khoản
#1	Vũ Quang Huy	vuquanghuy@gmail.com	User	Xóa tài khoản

Hình 4.3: Giao diện Quản lý Tài khoản người dùng của Quản trị viên

4.2.4 Phân hệ Kiểm duyệt Lịch trình (Dành cho Admin)

Để đảm bảo môi trường chia sẻ thông tin lành mạnh, hệ thống cung cấp cho Admin công cụ giám sát toàn bộ lịch trình cộng đồng. Admin có thể xem được thông tin người tạo, thời gian hành trình, thực hiện **Xem thử** nội dung chi tiết và **Gỡ bỏ (Xóa)** ngay lập tức nếu phát hiện lịch trình chứa nội dung không phù hợp.

	Dashboard	Danh mục	Địa điểm	Lịch trình của tôi	Kiểm duyệt Lịch trình	Quản Trị Viên Cao Cấp ▾
---	-----------	----------	----------	--------------------	------------------------------	-------------------------

Kiểm duyệt Lịch trình Cộng đồng			
TÊN LỊCH TRÌNH	NGƯỜI TẠO	THỜI GIAN	KIỂM DUYỆT
Hải Phòng 2 ngày 1 đêm	Nguyễn Văn A usera@gmail.com	23/05/2026 - 24/05/2026	Xem thử Gỡ bỏ (Xóa)
Lên Phố	Huy huyvu@gmail.com	23/05/2026 - 27/05/2026	Xem thử Gỡ bỏ (Xóa)
Hà Nội 3 ngày 2 đêm	Huy huyvu@gmail.com	22/05/2026 - 24/05/2026	Xem thử Gỡ bỏ (Xóa)
Hà Nội 3 ngày 2 đêm	Vũ Quang Huy vuquanghuy@gmail.com	22/05/2026 - 24/05/2026	Xem thử Gỡ bỏ (Xóa)

Hình 4.4: Giao diện Kiểm duyệt Lịch trình Cộng đồng của Quản trị viên

4.3 Đánh giá ưu và nhược điểm

- **Ưu điểm:** Giao diện trực quan, dễ sử dụng. Hệ thống phân quyền Admin/User chặt chẽ và không có lỗ hổng IDOR. Các tiện ích mở rộng như xuất PDF và tích hợp bản đồ mang lại giá trị thực tiễn cao cho trải nghiệm du lịch.
- **Nhược điểm:** Chưa tích hợp tính năng Trí tuệ nhân tạo (AI) để tự động gợi ý lịch

trình. Bộ máy tìm kiếm hiện tại mới dừng ở mức truy vấn chuỗi cơ bản, cần nâng cấp lên Full-text Search để tối ưu khi lượng dữ liệu lớn.

4.4 Phân công và đánh giá mức độ hoàn thành công việc

Nhóm gồm 3 thành viên, khối lượng công việc được chia đều dựa trên mức độ phức tạp của các module lập trình và biên soạn tài liệu.

Bảng 4.1: Bảng phân công nhiệm vụ và đánh giá đóng góp

STT	Họ và tên / MSSV	Nhiệm vụ đảm nhiệm	Đóng góp
1	Vũ Quang Huy (23016870 - Trưởng nhóm)	Thiết kế CSDL. Xây dựng Module Lịch trình, Pivot table, Tích hợp Google Maps và Triển khai server (Deploy).	33.3%
2	Vũ Thị Diệu Linh (23015647)	Khởi tạo dự án. Viết Module Authentication, Middleware phân quyền. Thiết kế giao diện Frontend (Tailwind).	33.3%
3	Nguyễn Ngọc Linh (23015739)	Xây dựng CRUD Module Địa điểm, Danh mục. Chức năng xuất file PDF. Soạn thảo báo cáo nghiên cứu.	33.4%
Tổng cộng:			100%

KẾT LUẬN VÀ KIẾN NGHỊ

1. Kết luận chung

Đề tài "**Phân tích, thiết kế và xây dựng Hệ thống quản lý lịch trình du lịch cá nhân**" đã hoàn thành tốt các mục tiêu đặt ra. Việc áp dụng kiến trúc MVC và framework Laravel không chỉ giúp giảm thiểu thời gian phát triển mà còn tạo ra một bộ mã nguồn (source code) sạch, dễ dàng bảo trì. Hệ thống đã giải quyết trọn vẹn bài toán xây dựng lộ trình (timeline) trực quan cho người dùng, kết hợp các tiện ích xuất bản (PDF, Public Link) và cơ chế quản trị mạnh mẽ.

2. Định hướng phát triển tương lai

Để nâng cấp ứng dụng trở thành một nền tảng quy mô thương mại, nhóm nghiên cứu đề xuất các hướng phát triển sau:

- **Gợi ý thông minh (Recommendation System):** Áp dụng Machine Learning để phân tích hành vi người dùng, từ đó tự động gợi ý lịch trình hoặc các địa điểm gần nhất (Location-based services).
- **Mạng xã hội du lịch:** Nâng cấp tính năng Public Link thành một diễn đàn thu nhỏ, cho phép người dùng khác sao chép (clone) lịch trình, bình luận và đánh giá chuyến đi.
- **Mobile App Native:** Cung cấp các API RESTful để phát triển thêm ứng dụng Native trên hệ điều hành Android và iOS.

TÀI LIỆU THAM KHẢO

- [1] Buhalis, D., & Amaranggana, A. (2014). *Smart tourism destinations*. Information and communication technologies in tourism 2014, 553-564.
- [2] Fesenmaier, D. R., Wöber, K. W., & Werthner, H. (2006). *Travel destination recommendation systems: Behavioral foundations and applications*. CABI.
- [3] Leff, A., & Rayfield, J. T. (2001). *Web-application development using the Model/View/Controller design pattern*. Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference, 118-127.
- [4] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). *Role-based access control models*. IEEE computer, 29(2), 38-47.
- [5] Otwell, T. (2023). *Laravel Documentation - The PHP Framework for Web Artisans*. [Trực tuyến]. Có tại: <https://laravel.com/docs>